

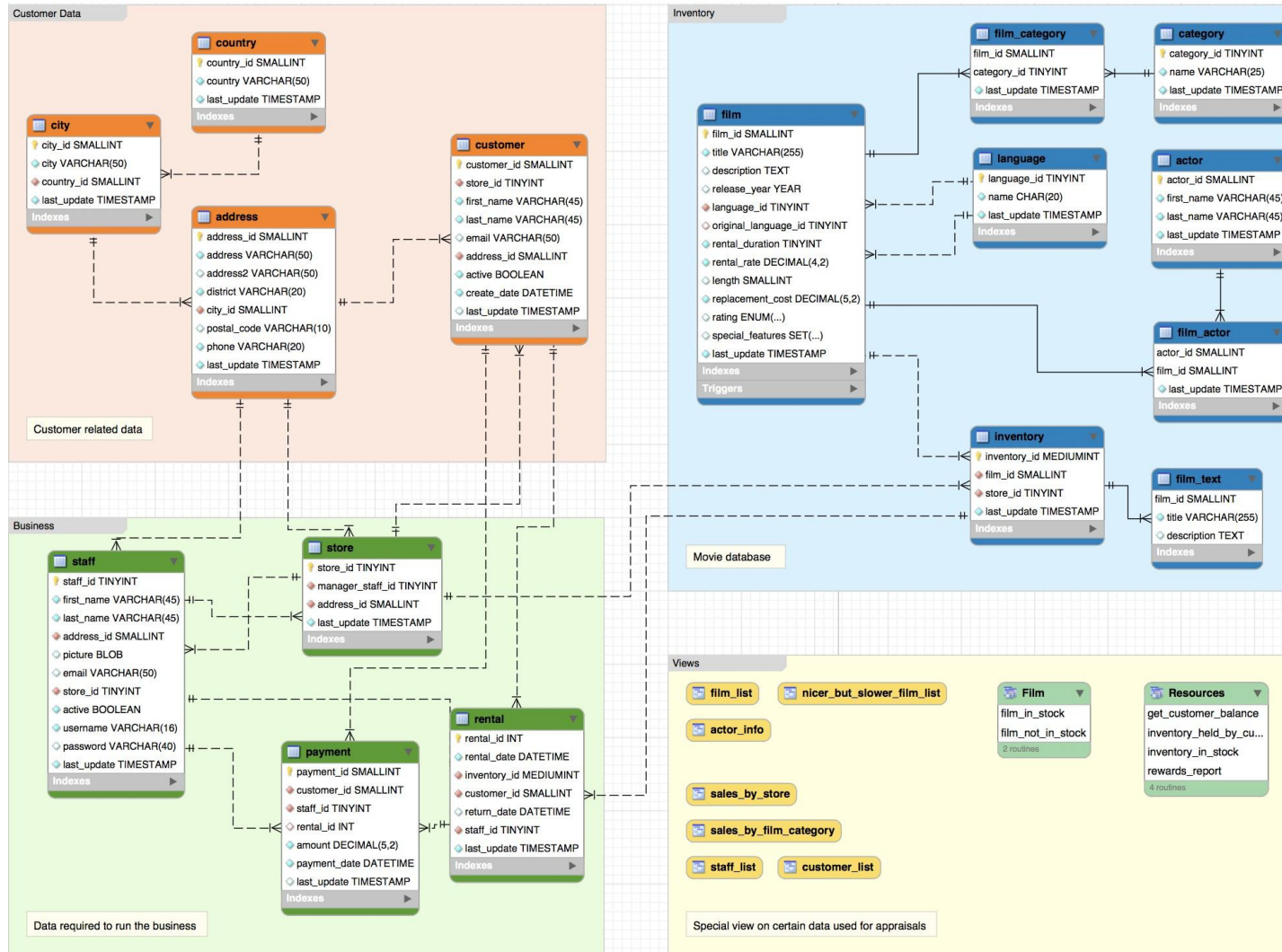
WHY LEARN SQL?

- ★ SQL is the **standard language** for relational database management
- ★ You can **load, query** and **analyze massive data sets** using SQL
- ★ Learning SQL is **fun, intuitive** and **surprisingly easy**
- ★ Companies **always** need employees who know SQL
- ★ Becoming a database + SQL expert has **never-ending practical benefits**

COMMON FLAVORS OF SQL



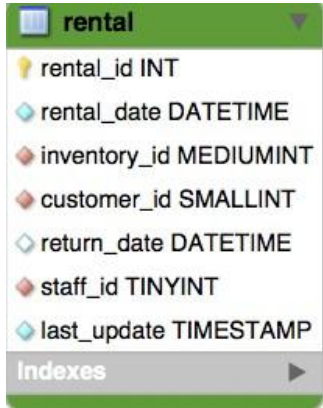
DATABASE CAN CONTAIN MANY RELATED TABLES



In this case our database includes **16 related tables**, containing information about:

- **Customers** (Name, Address, etc.)
- **Business** (Staff, Rentals, etc.)
- **Inventory** (Films, Categories, etc.)

EACH TABLE CONTAINS ROWS & COLUMNS

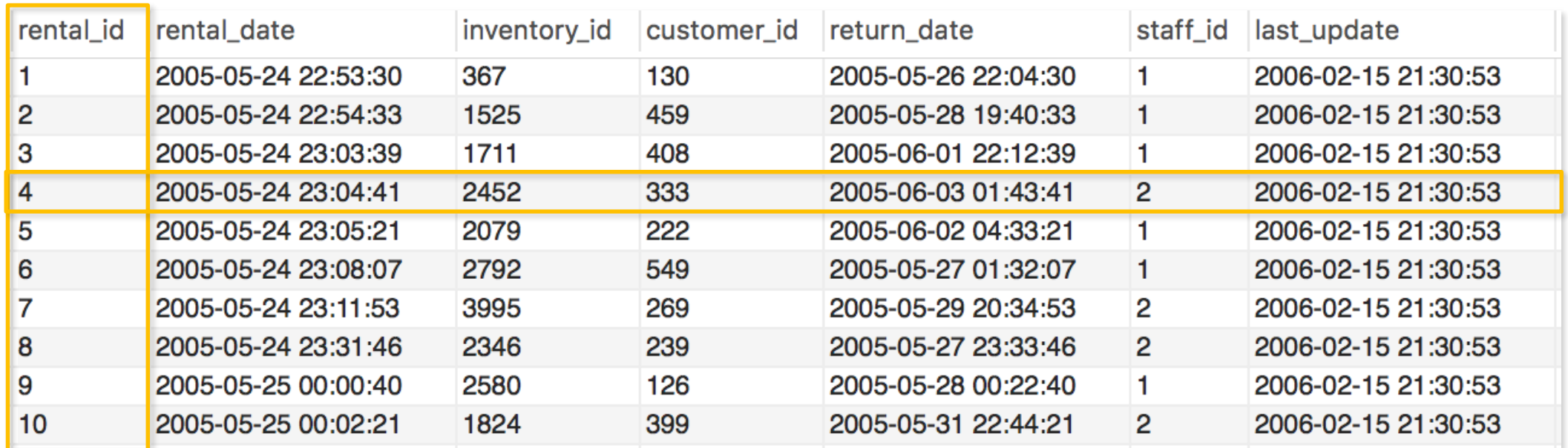


rental						
rental_id	INT					
rental_date	DATETIME					
inventory_id	MEDIUMINT					
customer_id	SMALLINT					
return_date	DATETIME					
staff_id	TINYINT					
last_update	TIMESTAMP					
Indexes						

Tables contain information organized into **columns** (*or fields*) and **rows** (*or records*)

In this case, our **Rental** table contains **7 columns** and **10 rows**:

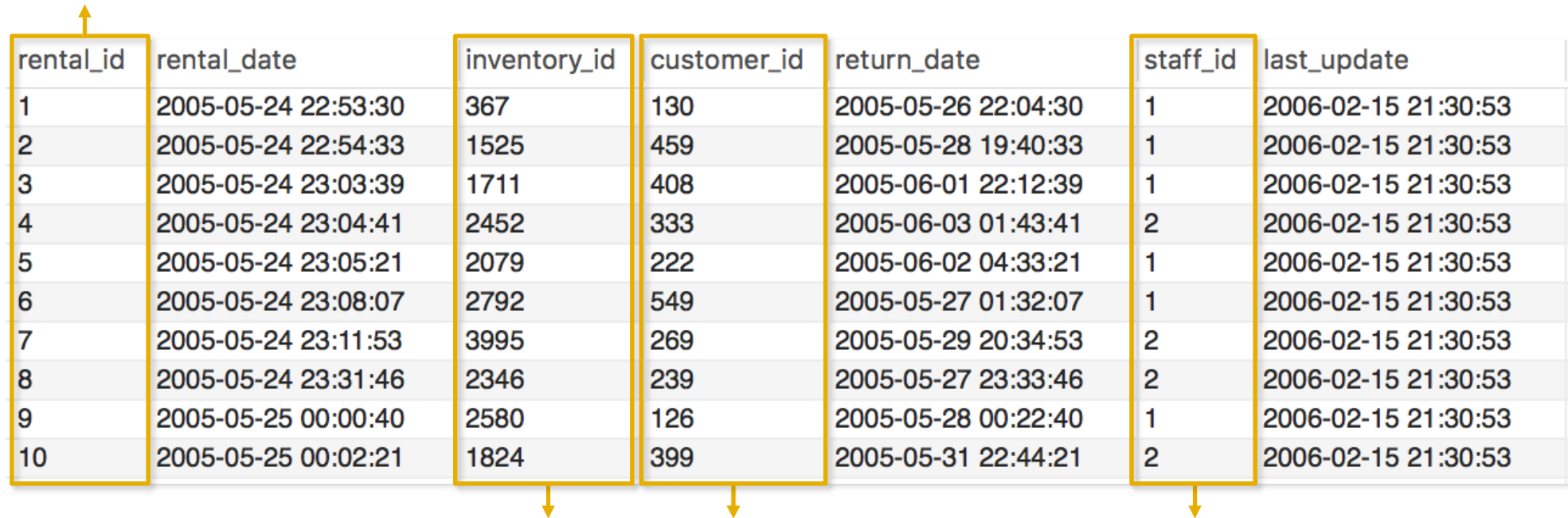
- Each **column** contains an attribute related to our film rentals (*rental/return date, customer ID, etc.*)
- Each **row** corresponds to **one specific rental** (*which film was rented, when, who rented it, etc.*)



rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update
1	2005-05-24 22:53:30	367	130	2005-05-26 22:04:30	1	2006-02-15 21:30:53
2	2005-05-24 22:54:33	1525	459	2005-05-28 19:40:33	1	2006-02-15 21:30:53
3	2005-05-24 23:03:39	1711	408	2005-06-01 22:12:39	1	2006-02-15 21:30:53
4	2005-05-24 23:04:41	2452	333	2005-06-03 01:43:41	2	2006-02-15 21:30:53
5	2005-05-24 23:05:21	2079	222	2005-06-02 04:33:21	1	2006-02-15 21:30:53
6	2005-05-24 23:08:07	2792	549	2005-05-27 01:32:07	1	2006-02-15 21:30:53
7	2005-05-24 23:11:53	3995	269	2005-05-29 20:34:53	2	2006-02-15 21:30:53
8	2005-05-24 23:31:46	2346	239	2005-05-27 23:33:46	2	2006-02-15 21:30:53
9	2005-05-25 00:00:40	2580	126	2005-05-28 00:22:40	1	2006-02-15 21:30:53
10	2005-05-25 00:02:21	1824	399	2005-05-31 22:44:21	2	2006-02-15 21:30:53

TABLES CAN CONTAIN PRIMARY & FOREIGN KEYS

The ***rental_id*** column is known as a **Primary Key**, which serves as a **unique identifier** for each record in the rental table



The diagram shows a table with 7 columns: rental_id, rental_date, inventory_id, customer_id, return_date, staff_id, and last_update. The rental_id column is highlighted with an orange box and an upward arrow, indicating it is the primary key. The inventory_id, customer_id, and staff_id columns are each highlighted with an orange box and a downward arrow, indicating they are foreign keys. The return_date and last_update columns are not highlighted.

rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update
1	2005-05-24 22:53:30	367	130	2005-05-26 22:04:30	1	2006-02-15 21:30:53
2	2005-05-24 22:54:33	1525	459	2005-05-28 19:40:33	1	2006-02-15 21:30:53
3	2005-05-24 23:03:39	1711	408	2005-06-01 22:12:39	1	2006-02-15 21:30:53
4	2005-05-24 23:04:41	2452	333	2005-06-03 01:43:41	2	2006-02-15 21:30:53
5	2005-05-24 23:05:21	2079	222	2005-06-02 04:33:21	1	2006-02-15 21:30:53
6	2005-05-24 23:08:07	2792	549	2005-05-27 01:32:07	1	2006-02-15 21:30:53
7	2005-05-24 23:11:53	3995	269	2005-05-29 20:34:53	2	2006-02-15 21:30:53
8	2005-05-24 23:31:46	2346	239	2005-05-27 23:33:46	2	2006-02-15 21:30:53
9	2005-05-25 00:00:40	2580	126	2005-05-28 00:22:40	1	2006-02-15 21:30:53
10	2005-05-25 00:02:21	1824	399	2005-05-31 22:44:21	2	2006-02-15 21:30:53

inventory_id, ***customer_id***, and ***staff_id*** are **Foreign Keys**, which **reference primary keys** from other tables

- Primary Keys ***cannot*** contain duplicates, and ***cannot*** be NULL; Foreign Keys ***can*** repeat, and ***can*** be NULL

THE “BIG 6” ELEMENTS OF A SQL SELECT STATEMENT

SELECT

Identifies the column(s) you want your query to select for your results

SELECT columnName

FROM

Identifies the table(s) your query will pull data from

FROM tableName

WHERE

(Optional) Specifies record-filtering criteria for filtering your results

WHERE logicalCondition

GROUP BY

(Optional) Specifies how to group the data in your results

GROUP BY columnName

HAVING

(Optional) Specifies group-filtering criteria for filtering your results

HAVING logicalCondition

ORDER BY

(Optional) Specifies the order in which your query results are displayed

ORDER BY columnName

SELECTING COLUMNS

- **SELECT * FROM** tells SQL to select *all* columns from
- You can return one or more specific columns in your results by naming them in your **SELECT** statement
- When you use **SELECT DISTINCT**, your result set will return just the **distinct** (*or unique*) values in those columns

```
SELECT * FROM Rental
```

```
SELECT  
    customer_id,  
    rental_date  
FROM rental;
```

```
SELECT DISTINCT  
    rating  
FROM film
```

ASSIGNMENT:

“I’m going to send an email letting our customers know there has been a management change.

*Could you pull a list of the **first name**, **last name**, and **email** of each of our customers?”*

Result Preview

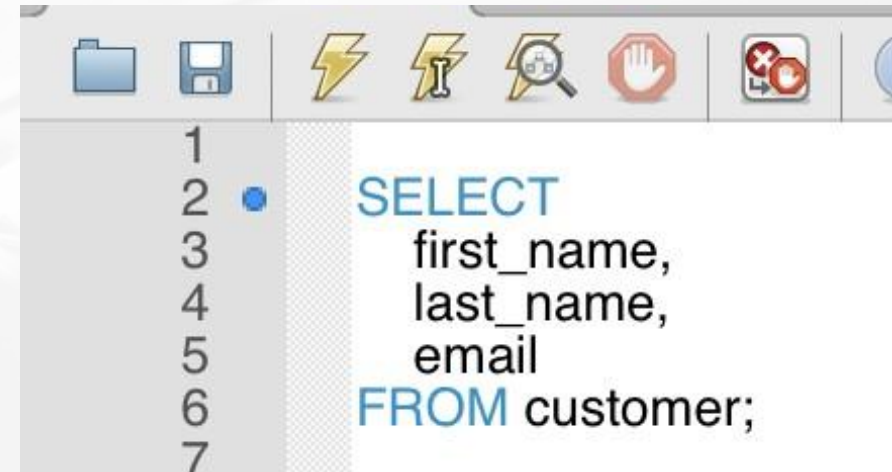
Result Grid			Filter Rows:	Search	Expo
	first_name	last_name	email		
►	MARY	SMITH	MARY.SMITH@sakilacustomer.org		
	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org		
	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org		
	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org		
	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org		
	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org		
	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org		
	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org		
	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org		
	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org		
	LISA	ANDERS...	LISA.ANDERSON@sakilacustomer.org		
	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org		
	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org		
	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org		
	HELEN	HARRIS	HELEN.HARRIS@sakilacustomer.org		
	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.org		

Answer

"I'm going to send an email letting our customers know there has been a management change.

*Could you pull a list of the **first name**, **last name**, and **email** of each of our customers?"*

Solution Query



The screenshot shows a SQL query editor window. The toolbar at the top includes icons for file operations (folder, save), execution (lightning bolt, lightning bolt with cursor), search (magnifying glass), and other functions (stop, refresh). On the left, a line editor shows line numbers 1 through 7, with line 2 selected. The SQL query text is as follows:

```
SELECT
first_name,
last_name,
email
FROM customer;
```

ASSIGNMENT:

*“My understanding is that we have titles that we rent for durations of **3, 5, or 7** days.*

*Could you pull the records of our films and see if there are any other **rental durations?**”*

Result Preview

Result Grid		
	rental_duration	
▶	6	
	3	
	7	
	5	
	4	

Answer:

*“My understanding is that we have titles that we rent for durations of **3, 5, or 7** days.*

*Could you pull the records of our films and see if there are any other **rental durations**?”*

Solution Query

```
SELECT DISTINCT  
    rental_duration  
FROM film
```

WHERE CLAUSE

WHERE

(Optional) Specifies criteria for filtering the records of your result set

This is where you prescribe the logical conditions used to filter your result set

- **WHERE** category = 'Sci-Fi'
- **WHERE** amount > 5.99
- **WHERE** rental_date BETWEEN '2006-01-01' AND '2006-06-01'

Pulling records **before or after a date**, or **between two dates**

Limiting your result set to include only transactions **above a certain payment amount**

Filtering your result set to just records **related to certain customers**, or to **certain lines of business**

WHERE OPERATOR

Operator	What it Means
=	Equals
<>	Does NOT Equal
>	Greater Than
<	Less Than
>=	Greater Than Or Equal To
<=	Less Than Or Equal To
BETWEEN	A Range Between Two Values
LIKE	Matching a Pattern Like This
IN()	Equals One of These Values

```
SELECT
    customer_id,
    rental_id,
    amount,
    payment_date
FROM payment
WHERE amount = 0.99
```

```
SELECT
    customer_id,
    rental_id,
    amount,
    payment_date
FROM payment
WHERE payment_date > '2006-01-01'
```

ASSIGNMENT:

*“I’d like to look at **payment records** for our **long-term customers** to learn about their purchase patterns.*

*Could you pull all **payments** from our **first 100 customers** (based on customer ID)?”*

Result Preview

Result Grid				Filter Rows:	Search
customer_id	rental_id	amount	payment_date		
1	76	2.99	2005-05-25 11:30:37		
1	573	0.99	2005-05-28 10:35:23		
1	1185	5.99	2005-06-15 00:54:12		
1	1422	0.99	2005-06-15 18:02:53		
1	1476	9.99	2005-06-15 21:08:46		
1	1725	4.99	2005-06-16 15:18:57		
1	2308	4.99	2005-06-18 08:41:48		
1	2363	0.99	2005-06-18 13:33:59		
1	3284	3.99	2005-06-21 06:24:45		
1	4526	5.99	2005-07-08 03:17:05		
1	4611	5.99	2005-07-08 07:33:56		

WHERE CLAUSES

Answer:

*“I’d like to look at **payment records** for our **long-term customers** to learn about their purchase patterns.*

*Could you pull all **payments** from our **first 100 customers** (based on customer ID)?”*

Solution Query

```
SELECT
  customer_id,
  rental_id,
  amount,
  payment_date
FROM payment
WHERE customer_id < 101;
```

ALTERNATIVE OPTIONS:

```
WHERE customer_id <= 100
```

```
WHERE customer_id BETWEEN 1 AND 100
```

WHERE CLAUSES

WHERE & AND:

- You can include **multiple logical conditions** in your **WHERE** clause using an **AND** statement
- Use **AND** to return records which satisfy *all* criteria

```
SELECT
  customer_id,
  rental_id,
  amount,
  payment_date
FROM payment
WHERE amount = 0.99
AND payment_date > '2006-01-01'
```

QUERY RESULTS:

Result Grid

Filter Rows:

	customer_id	rental_id	amount	payment_date
▶	5	13209	0.99	2006-02-14 15:16:03
	11	11646	0.99	2006-02-14 15:16:03
	29	15577	0.99	2006-02-14 15:16:03
	33	12277	0.99	2006-02-14 15:16:03
	58	15326	0.99	2006-02-14 15:16:03
	69	11995	0.99	2006-02-14 15:16:03
	99	11593	0.99	2006-02-14 15:16:03
	100	15021	0.99	2006-02-14 15:16:03

ASSIGNMENT:

“The payment data you gave me on our first 100 customers was great – thank you!

*Now I’d love to see **just payments over \$5** for those same customers, since January 1, 2006.”*

Result Preview

Result Grid				Filter Rows:		Search
customer_id	rental_id	amount	payment_date			
42	13351	5.98	2006-02-14 15:16:03			
53	11657	7.98	2006-02-14 15:16:03			
60	12489	9.98	2006-02-14 15:16:03			
75	13534	8.97	2006-02-14 15:16:03			

WHERE & AND

ANSWER:

“The payment data you gave me on our first 100 customers was great – thank you!

*Now I’d love to see **just payments over \$5 for those same customers, since January 1, 2006.**”*

Solution Query

```
SELECT
  customer_id,
  rental_id,
  amount,
  payment_date
FROM payment
WHERE customer_id < 101
  AND amount > 5
  AND payment_date > '2006-01-01';
```

ALTERNATIVE OPTIONS:

*You can add the three filters in any order without changing the outcome, and can define the **customer_id** filter in a number of different ways.*

As long as you see the 4 records shown on the previous slide, you’re good to go!

WHERE & AND

WHERE & OR

- You can include **multiple logical conditions** in your **WHERE** clause using an **OR** statement
- Use **OR** to return records which satisfy *any* criteria

```
SELECT
    customer_id,
    rental_id,
    amount,
    payment_date
FROM payment
WHERE customer_id = 5
    OR customer_id = 11
    OR customer_id = 29;
```

QUERY RESULTS:

Result Grid				Filter Rows:	Search
	customer_id	rental_id	amount	payment_date	
▶	5	731	0.99	2005-05-29 07:25:16	
	5	1085	6.99	2005-05-31 11:15:43	
	5	1142	1.99	2005-05-31 19:46:38	
	5	1502	3.99	2005-06-15 22:03:14	
	5	1631	2.99	2005-06-16 08:01:02	
	5	2063	4.99	2005-06-17 15:56:53	
	5	2570	2.99	2005-06-19 04:20:13	

ASSIGNMENT:

“The data you shared previously on customers 42, 53, 60, and 75 was good to see.

*Now, could you please write a query to pull **all payments from those specific customers, along with payments over \$5, from any customer?**”*

Result Preview

Result Grid				Filter Rows:	Search
customer_id	rental_id	amount	payment_date		
1	1185	5.99	2005-06-15 00:54:12		
1	1476	9.99	2005-06-15 21:08:46		
1	4526	5.99	2005-07-08 03:17:05		
1	4611	5.99	2005-07-08 07:33:56		
1	6163	7.99	2005-07-11 10:13:46		
1	15315	5.99	2005-08-22 20:03:46		
2	5755	6.99	2005-07-10 12:38:56		
2	7376	5.99	2005-07-27 15:23:02		
2	7459	5.99	2005-07-27 18:40:20		
2	8230	5.99	2005-07-29 00:12:59		

WHERE & OR

ANSWER

“The data you shared previously on customers 42, 53, 60, and 75 was good to see.

*Now, could you please write a query to pull **all payments from those specific customers**, along with **payments over \$5**, from any customer?”*

Solution Query

```
SELECT
    customer_id,
    rental_id,
    amount,
    payment_date
FROM payment
WHERE amount > 5
    OR customer_id = 42
    OR customer_id = 53
    OR customer_id = 60
    OR customer_id = 75
```

ALTERNATIVE OPTIONS:

The logic in this WHERE can be written in any order.

WHERE & OR

WHERE & IN

- If you find yourself writing multiple **OR** conditions that reference different values in the *same column*, you can use **IN()** to save some time
- The two queries shown at the right produce identical results, but the second version is easier to read (*and write!*)
- *Using **IN** saves time, and will make it easier to read your query later*

```
SELECT
  customer_id,
  rental_id,
  amount,
  payment_date
FROM payment
WHERE amount > 5
OR customer_id = 42
OR customer_id = 53
OR customer_id = 60
OR customer_id = 75
```

These two queries do the exact same thing

```
SELECT
  customer_id,
  rental_id,
  amount,
  payment_date
FROM payment
WHERE amount > 5
OR customer_id IN (42,53,60,75)
```

GROUP BY CLAUSE

GROUP BY

(Optional) Specifies how to group the data in your results like Pivot

GROUP BY is great for comparing different segments of your data (similar to Pivot Tables in Excel)

- **GROUP BY** category
- **GROUP BY** store_location
- **GROUP BY** store_location, category

GROUP AGGREGATION

GROUP BY allows us to perform segment analysis on our data at various levels of granularity

Combine **GROUP BY** with **aggregate functions** (like **COUNT** or **SUM**) to specify how values are summarized for each group

```
SELECT  
  rating,  
  COUNT(film_id)  
FROM film  
GROUP BY  
  rating
```

COMMENTS & ALIASES

- **Comments** can be added to your code using “--” or “/*”, which tells SQL to ignore those lines
 - Comments can apply to **entire lines**, **portions of a line**, or **multiple lines**
- **Aliases** allow you to assign a custom name to a field in your result set, using an **AS** statement

ASSIGNMENT:

“I need to get a quick overview of how long our movies tend to be rented out for.

*Could you please pull a **count of titles sliced by rental duration?**”*

Result Preview

Result Grid				Filter Rows:	Search
rental_duration	films_with_this_rental_duration				
6	212				
3	203				
7	191				
5	191				
4	203				

GROUP BY

ANSWER

“I need to get a quick overview of how long our movies tend to be rented out for.

*Could you please pull a **count of titles sliced by rental duration?**”*

Solution Query

```
SELECT
    rental_duration,
    COUNT(film_id) AS films_with_this_rental_duration
FROM film
GROUP BY
    rental_duration
```

GROUP BY

MULTIPLE GROUP BY

- You can use **GROUP BY** with *multiple* columns at once, by listing the columns and separating them with a comma
- This allows you to create groups and sub-groups in your result set, *(just like specifying **multiple row or column labels** in a PivotTable!)*

```
SELECT
  rating,
  rental_duration,
  COUNT(film_id) AS count_of_films
FROM film
GROUP BY
  rating,
  rental_duration
```

QUERY RESULTS:

Result Grid		Filter Rows:
rating	rental_duration	count_of_films
PG	6	39
G	3	49
NC-17	7	40
G	5	33
G	6	39
PG	3	36
PG-13	6	50
R	6	27
PG-13	3	39
NC-17	6	57

AGGREGATE FUNCTIONS

- The powerful functions below can all be used with **GROUP BY** to provide group-level summaries

Function	Purpose
COUNT()	Count of Records <i>Skips NULL, except COUNT(*)</i>
COUNT DISTINCT()	Count of Distinct Values <i>Skips NULL values</i>
MIN()	Finds the Smallest Value <i>Skips NULL values</i>
MAX()	Finds the Largest Value <i>Skips NULL values</i>
AVG()	Average of All Values <i>Skips NULL values</i>
SUM()	SUM of All Values <i>Treats NULL values as Zero</i>

```
SELECT
  rating,
  COUNT(film_id) AS count_of_films,
  MIN(length) AS shortest_film,
  MAX(length) AS longest_film,
  AVG(length) AS average_length_of_film,
  -- SUM(length) AS total_minutes
  AVG(rental_duration) AS average_rental_duration

FROM film
GROUP BY
  rating
```

QUERY RESULTS:

Result Grid			Filter Rows:	Search	Export: 
rating	count_of_films	shortest_film	longest_film	average_length_of_film	average_rental_duration
PG	194	46	185	112.0052	5.0825
G	178	47	185	111.0506	4.8371
NC-17	210	46	184	113.2286	5.1429
PG-13	223	46	185	120.4439	5.0538
R	195	49	185	118.6615	4.7744

ASSIGNMENT:

“I’m wondering if we charge more for a rental when the replacement cost is higher.

*Can you help me pull a **count of films**, along with the **average**, **min**, and **max rental rate**, grouped **by replacement cost**?”*

Result Preview

Result Grid		Filter Rows:	Search	Export:
replacement_cost	number_of_films	cheapest_rental	most_expensive_rental	average_rental
29.99	53	0.99	4.99	2.537170
28.99	41	0.99	4.99	2.990000
27.99	53	0.99	4.99	2.839057
26.99	46	0.99	4.99	2.990000
25.99	43	0.99	4.99	2.850465
24.99	38	0.99	4.99	3.147895
23.99	45	0.99	4.99	2.723333
22.99	55	0.99	4.99	3.062727
21.99	55	0.99	4.99	2.990000
20.99	57	0.99	4.99	2.919825
19.99	50	0.99	4.99	3.110000

AGGREGATE FUNCTIONS

ANSWER:

“I’m wondering if we charge more for a rental when the replacement cost is higher.

*Can you help me pull a **count of films**, along with the **average, min, and max rental rate**, grouped **by replacement cost**?”*

Solution Query

```
SELECT
  replacement_cost,
  COUNT(film_id) AS number_of_films,
  MIN(rental_rate) AS cheapest_rental,
  MAX(rental_rate) AS most_expensive_rental,
  AVG(rental_rate) AS average_rental
FROM film
GROUP BY
  replacement_cost
```

AGGREGATE FUNCTIONS

HAVING CLAUSE

HAVING

(Optional) Specifies group-filtering criteria for filtering your results

- **HAVING** is an optional clause you can use with **GROUP BY** to limit your result set to groups which satisfy certain logical criteria
- **HAVING** comes *after* the **GROUP BY** clause and *before* **ORDER BY**, if you are sorting your results (*we'll cover this next*)
- **HAVING** is a great way to limit your results to the most important groups for your business. For example, you can limit results to customers with total payments above a certain amount, or to your most-rented films

Examples:

- **HAVING COUNT(*) > 1**
- **HAVING SUM(payment) > 10**
- **HAVING MIN(rental_date) < '2005-05-25'**

ASSIGNMENT:

“I’d like to talk to customers that have not rented much from us to understand if there is something we could be doing better.

*Could you pull a list of **customer_ids** with **less than 15 rentals** all-time?”*

Result Preview

Result Grid				Filter
	customer_id	total_rentals		
▶	61	14		
	110	14		
	281	14		
	318	12		

HAVING

ANSWER:

“I’d like to talk to customers that have not rented much from us to understand if there is something we could be doing better.

*Could you pull a list of **customer_ids** with **less than 15 rentals** all-time?”*

Solution Query

```
SELECT
    customer_id,
    COUNT(*) AS total_rentals
FROM rental
GROUP BY
    customer_id
HAVING
    COUNT(*) < 15
```

HAVING

ORDER BY CLAUSE

ORDER BY

(Optional) Specifies the order in which your query results are displayed

- SQL lets you sort your results by including **ORDER BY** after your **FROM** clause (and after your **WHERE**, **GROUP BY**, and **HAVING** clauses, if applicable)
- **ORDER BY** defaults to **ascending** order (*low to high*), but can be modified to sort in **descending** order by adding **DESC** after the column reference

Examples:

- **ORDER BY** amount
- **ORDER BY** created_date **DESC**
- **ORDER BY** created_date **DESC**, amount **DESC**

ASSIGNMENT:

“I’d like to see if our longest films also tend to be our most expensive rentals.

*Could you pull me a list of **all film titles** along with their **lengths** and **rental rates**, and **sort them from longest to shortest**?”*

Result Preview

Result Grid			Filter Rows:
title	length	rental_rate	
CHICAGO NORTH	185	4.99	
CONTROL ANTHEM	185	4.99	
DARN FORRESTER	185	4.99	
GANGS PRIDE	185	2.99	
HOME PITY	185	4.99	
MUSCLE BRIGHT	185	2.99	
POND SEATTLE	185	2.99	
SOLDIERS EVOLUTION	185	4.99	

ORDER BY

ANSWER:

“I’d like to see if our longest films also tend to be our most expensive rentals.

*Could you pull me a list of **all film titles** along with their **lengths** and **rental rates**, and **sort them from longest to shortest**?”*

Solution Query

```
SELECT  
  title,  
  length,  
  rental_rate  
FROM film  
ORDER BY length DESC;
```

ORDER BY